LESSON 1 · COMPUTER FUNDAMENTALS TRACK

How Computers Actually Work

The missing foundation behind the Python you already know

You've learned Python basics — variables, loops, functions, maybe even a few small projects. But there's a layer underneath all of that which most self-taught programmers skip entirely: **what is actually happening inside the machine** when your code runs. This lesson fills that gap. It won't make you write better Python syntax — it will make you understand *why* Python behaves the way it does, and give you the vocabulary every serious developer is expected to know.

> **Why this lesson exists**
>
> Knowing Python without knowing how computers work is like knowing how to drive without knowing what an engine does. You can get by for a while — until something breaks, something runs slowly, or someone asks you a question you can't answer. This track builds that missing layer, one concept at a time.

What you'll walk away with

- A clear mental model of the four core hardware components inside any computer.
- How all data — text, images, numbers — is really just patterns of 1s and 0s, and how encoding turns those patterns into meaning.
- What actually happens between you typing `python app.py` and seeing output on screen — including what bytecode and the Python Virtual Machine are.
- How Python specifically stores and manages your variables in memory, and why some bugs happen because of this.
- How your operating system juggles many programs on limited hardware.
- The vocabulary to describe all of this clearly and confidently — in interviews, in code reviews, and in your own debugging.

Session Plan — 2 Hour Class

This lesson is built to run as a single 2-hour, in-person session. Suggested pacing below — adjust on the day based on how discussion is flowing. Detailed talking points, extra examples, and discussion prompts for each segment are included in the sections that follow, so you have more than enough material to fill the time.

Time	Segment	Focus

5 min	Welcome & framing	Why this lesson matters, what you'll walk away with
15 min	§1 Hardware basics	CPU, RAM, storage, I/O, buses — the components and how they talk
20 min	§2 Binary & encoding	Bits, bytes, place values, ASCII/Unicode, hands-on demo
10 min	Break	—
20 min	§3 Code to execution	Source → bytecode → PVM → machine code, compiled vs. interpreted, dis module
20 min	§4 Variables & memory	References, id(), mutability, stack vs. heap, prediction exercise
15 min	§5 Operating system	Processes, threads, scheduling, context switching, the GIL
15 min	Wrap-up & Q&A;	Key terms recap, homework, next lesson preview

■ 15 MIN

1. What Is a Computer, Really?

Strip away the screen, the keyboard, and the operating system, and every computer — your laptop, your phone, a server in a data center — is built from four core parts working together. Understanding these isn't trivia; it directly explains things you've probably already experienced, like why a program "runs out of memory," why an SSD makes a laptop feel faster than more RAM would, or why some tasks are CPU-bound while others are I/O-bound.

Component	Job	Everyday analogy
CPU (Processor)	Executes instructions — the actual "thinking"	The chef, doing the cooking
RAM (Memory)	Holds data the CPU is actively working with, right now	The kitchen counter — fast, but cleared once you're done
Storage (Disk/SSD)	Holds data permanently, even when powered off	The pantry — slower to reach, but nothing disappears
I/O (Input/Output)	Moves data in and out — keyboard, screen, network, disk	Waiters carrying orders in and dishes out

Here's the sequence that matters most for you as a programmer: when your Python program runs, your code and the data it uses get loaded into **RAM**. The **CPU** reads instructions from RAM one at a time, executes them, and writes results back to RAM. Nothing is permanently saved unless it's explicitly written to **storage** — which is exactly why closing a program without saving loses your work.

How the components actually talk to each other

These four parts aren't just sitting near each other — they're connected by a **bus**, a set of physical pathways that carry data back and forth. Think of it as the road network between the kitchen, counter, and pantry in our analogy. The speed of this connection matters: RAM is connected to the CPU by a very fast bus, which is why reading from RAM is thousands of times faster than reading from a disk, even a fast SSD. This speed gap is a big part of why programs are designed to keep frequently-used data in RAM rather than constantly reading from storage.

Putting real numbers to it

Specs on a typical laptop today might look like: a quad-core CPU, 8–16 GB of RAM, and 256 GB–1 TB of SSD storage. "Quad-core" means the CPU itself contains four independent processing units, each able to execute instructions — this is part of how a modern computer runs many programs smoothly instead of freezing on one task. When your Python script runs, it's typically using just one of those cores unless you specifically write code to use multiple ones (a topic for a later, more advanced lesson).

> **Quick check**
>
> If RAM is cleared every time you restart your computer, why doesn't your saved Python file disappear when you shut down? *(Answer: the file lives in storage, not RAM — it's only loaded into RAM while the program is actively running.)*

Discuss with your instructor

- Which of these four components would you upgrade first to make your Python programs run noticeably faster — and why? There isn't one universally correct answer; it depends on what kind of program you're running.
- Have you ever seen a "Not enough memory" or "Disk full" error? Which component was actually the bottleneck in each case?

■ 20 MIN

2. Everything Is Just Numbers: Binary

Computers don't understand letters, images, or Python keywords directly. At the lowest level, a computer only recognizes two electrical states: **on** and **off**. We represent these as **1** and **0** — this is called **binary**. Every single thing your computer stores or processes — this sentence, a photo, the number 7, a Python script — is ultimately a very long sequence of 1s and 0s.

The building blocks

- **Bit** — a single 1 or 0. The smallest unit of data a computer understands.
- **Byte** — a group of 8 bits. One byte can represent 256 different values (0–255).
- **Encoding** — an agreed-upon rule for turning bytes into meaningful data, like text. For example, ASCII/Unicode says the byte pattern for the number 65 represents the letter `'A'`.

How binary counting actually works

In our normal decimal system, each digit position represents a power of 10 (1, 10, 100...). Binary works the same way, but each position represents a power of 2 instead: 1, 2, 4, 8, 16, 32, 64, 128. To read a binary number, add up the place values where there's a 1.

128	64	32	16	8	4	2	1
0	1	0	0	0	0	0	1

*The row above reads `01000001` — add 64 + 1 = **65**. That's the exact binary pattern behind the character `'A'` you'll see in the demo below.*

```
>>> ord('A')      # get the number behind a character
65
>>> bin(65)        # see that number in binary
'0b1000001'
>>> chr(65)        # go back from number to character
'A'
>>> ord('a')       # lowercase letters are different numbers entirely
97
```

Try this in your own Python shell. It's a small demo, but it proves something big: the letter `'A'` you type on your keyboard is not stored as a letter at all — it's stored as the number 65, in binary.

ASCII vs. Unicode — why this matters for real projects

ASCII, the original encoding standard, only covers 128 characters — enough for English letters, digits, and basic punctuation, but nothing else. This is exactly why ASCII cannot represent characters used in Somali, Arabic, or emoji. **Unicode** was created to solve this: it assigns a unique number to every character in virtually every writing system in the world, plus symbols and emoji. Python 3 strings are Unicode by default, which is why you can write

and print Somali, Arabic, or emoji text directly in your scripts without special handling.

```
>>> ord('ñ')
241
>>> ord('■')
128512
>>> len('Salaam')     # 6 characters, 6 code points
6
```

> **Why this matters for you as a programmer**
>
> This is the reason a Python `int` and a `str` are fundamentally different types, even if they look similar on screen (`5` vs `"5"`). One is stored as a binary number ready for arithmetic; the other is stored as encoded character data. That's also why `"5" + "5"` gives `"55"` instead of `10` — Python is working with two completely different underlying representations.

Try it yourself (5 minutes)

- Find the binary value of your own first initial using `bin(ord(...))`.
- Guess, then check: is `ord('Z')` bigger or smaller than `ord('A')`? By how much?
- Try `ord()` on a Somali or Arabic character and see what number comes back.

■ 20 MIN

3. From Your Code to a Running Program

When you write `print("Hello")` and run it, your readable Python code has to be transformed into something the CPU can actually execute. Here's that journey, step by step.

Step 1 — Source code

You write human-readable Python in a .py file. This is for you, not the machine.

Step 2 — Interpretation

The Python interpreter (CPython, the standard implementation) reads your code and translates it into a lower-level form called bytecode.

Step 3 — Bytecode

An intermediate, simplified set of instructions — not quite machine code yet, but no longer human Python syntax either. This is what gets cached in the __pycache__ folder as .pyc files, so Python doesn't have to re-translate unchanged code every run.

Step 4 — Python Virtual Machine (PVM)

The PVM executes that bytecode line by line, actually carrying out your program's logic.

Step 5 — Machine code & CPU

Under the hood, the PVM itself is a program, ultimately running as machine code that the CPU executes directly, in binary.

This is exactly why Python is called an **interpreted language**, unlike C or Rust, which are **compiled** directly to machine code before you ever run them. It's also why Python programs are generally slower than compiled languages, and why you don't need a separate "build" step before running your script.

Compiled vs. interpreted, side by side

	Compiled (e.g. C, Rust)	Interpreted (e.g. Python)
Before running	Entire program translated to machine code once, ahead of time	Translated to bytecode as it runs, or just before
Speed	Generally faster — no translation overhead at runtime	Generally slower — translation happens during execution

Flexibility	Must recompile after every code change	Run immediately after any change — great for fast iteration
Errors caught	Many caught before the program ever runs	Often only caught when that exact line executes

See bytecode for yourself

Python actually lets you look at the bytecode your code compiles to, using the built-in `dis` (disassembler) module. This is a great live demo to run in class.

```
import dis

def add(a, b):
    return a + b

dis.dis(add)

# Output looks something like:
#  2           0 LOAD_FAST     0 (a)
#              2 LOAD_FAST     1 (b)
#              4 BINARY_ADD
#              6 RETURN_VALUE
```

Each of those lines is one bytecode instruction — this is the exact intermediate form your simple `a + b` *becomes before it ever reaches the CPU.*

■ 20 MIN

4. How Python Actually Stores Your Variables

Here's something that surprises most self-taught Python developers: in Python, a variable is not a labeled box that holds a value. It's a **name that points to an object living somewhere in memory**.

```
a = [1, 2, 3]
b = a           # b now points to the SAME list object as a
b.append(4)
print(a)         # [1, 2, 3, 4]  <- a changed too!
```

This trips up almost every intermediate learner at some point. Because `a` *and* `b` *both point to the same object in RAM rather than holding independent copies, changing the list through one name changes it for the other too. This is the concept of* ***references****, and it's a direct consequence of how memory actually works — not a Python quirk.*

Proving it with id()

Python gives you a way to see the actual memory reference behind any variable: the built-in `id()` function. Run this alongside the example above to make the concept concrete rather than abstract.

```
>>> a = [1, 2, 3]
>>> b = a
>>> id(a) == id(b)
True                # same object in memory
>>> c = [1, 2, 3]
>>> id(a) == id(c)
False               # equal VALUES, but different objects
>>> a is b
True
>>> a is c
False
```

This is also the difference between Python's `==` *(checks if values are equal) and* `is` *(checks if they're the exact same object in memory) — a distinction that only makes real sense once you understand references.*

Mutable vs. immutable

Not every type behaves like the list example above. Python types split into two groups:

Immutable (cannot change in place)	Mutable (can change in place)
int, float, str, tuple, bool	list, dict, set, and your own custom class instances (usually)
Reassigning creates a brand-new object	Modifying methods (like .append()) change the existing object in place

Stack vs. heap, in plain terms

- **The stack** — fast, organized memory used for simple, short-lived values and keeping track of function calls. Every time you call a function, a new "frame" is pushed onto the stack holding its local variables; when the function returns, that frame is popped off and discarded.
- **The heap** — larger, more flexible memory where more complex objects (like lists, dictionaries, and your own classes) actually live.
- When you write `a = [1, 2, 3]`, the list itself lives on the heap, and `a` is just a reference (an address) pointing to it.

> **Bring to your next meetup**
>
> Predict what this prints, before running it: `x = 10; y = x; y = y + 5; print(x, y)`. Then explain, in your own words, why numbers behave differently from lists here. This is the core distinction between mutable and immutable objects in Python — and it only makes sense once you understand references.

■ 15 MIN

5. The Operating System's Role

One more piece completes the picture: your computer is running many programs at once, but has a limited number of CPU cores. The **operating system** (Windows, macOS, Linux) is the manager that decides which program gets to use the CPU, and for how long, switching between them so fast it feels simultaneous.

- **Process** — a running instance of a program (e.g. your Python script while it's executing). Each process gets its own private slice of memory, isolated from other processes.
- **Thread** — a smaller unit of work within a process; a process can have several running "in parallel." Threads within the same process share memory with each other.
- **Scheduling** — the OS deciding, many times per second, which process gets CPU time next.
- **Context switching** — the act of the CPU pausing one process, saving its exact state, and loading another process's state so it can run instead. This happens so fast (milliseconds) that it feels like everything runs at once, even on a single core.

When you run `python app.py`, the OS creates a new process, asks the storage device to load your file, hands it to the CPU and RAM to execute, and manages everything running alongside it — your browser, your music player, background updates — all sharing the same physical hardware.

A note on Python threads: the GIL

If you go on to use Python's `threading` module, you'll eventually run into something called the **Global Interpreter Lock (GIL)** — a mechanism in CPython that only allows one thread to execute Python bytecode at a time, even on a multi-core machine. This is a direct consequence of everything we've covered today: since the PVM executes bytecode step by step, CPython uses the GIL to keep that process safe and predictable. It's a common interview topic and a real source of confusion for developers who expect Python threads to behave like threads in other languages — good to know it exists, even before you need to work around it.

See it for yourself

- Open Task Manager (Windows) or Activity Monitor (macOS) while a Python script is running and find it in the process list.
- Notice how many other processes are running at the same time on a machine that feels like it's doing "one thing" from your perspective.

6. Summary: The Full Journey

You type Python code (source) → the interpreter turns it into bytecode → the operating system creates a process and loads it into RAM → the CPU executes instructions from RAM, one at a time, in binary → results get displayed via output, and optionally saved to storage. Every program you will ever write, in any language, follows some version of this same journey.

Key terms from this lesson

CPU · RAM · Storage · I/O · Bus · Bit · Byte · Binary · Encoding (ASCII/Unicode) · Source code · Interpreter · Bytecode · Compiled vs. interpreted languages · Reference · id() · Mutable/immutable · Stack · Heap · Process · Thread · Scheduling · Context switching · GIL

Before your next meetup

- Run the `ord()` / `chr()` / `bin()` examples yourself and try a few other letters, numbers, and Somali or Arabic characters.
- Run the `dis.dis()` example on a function of your own and see what the bytecode looks like.
- Work through the "Bring to your next meetup" prediction exercise above and be ready to explain your reasoning.
- Write down, in your own words (2–3 sentences), what happens between running `python app.py` and seeing output appear. No need to be perfect — we'll refine it together.

Next lesson: How the Internet Actually Works — what really happens between typing a URL and a page loading on your screen.